## Task 1: Public Transport Route Fare API Integration

**Task:** Provide a Python script to retrieve public transport fare details between two stations. The script should feature AI-generated code, input validation, error handling, and structured output. Allow for the integration of a new, unspecified Public Transport API.

```
import requests
import json

# --- Configuration for ANOTHER API --- #
# !!! IMPORTANT: Replace with your actual NEW API endpoint and API key
!!!
NEW_API_ENDPOINT = "https://api.another-transport.example.com/fare"
NEW_API_KEY = "YOUR_NEW_API_KEY"

# --- Helper Functions (potentially adapted for the new API) --- #

def _validate_station_name_new_api(station_name: str) -> bool:
    """
    Placeholder for validating station names for the NEW API.
    This might involve querying a station lookup endpoint specific to
the new API
    or checking against a known list.
    """
    if not isinstance(station_name, str) or not station_name.strip():
        return False
    # For demonstration, 'UnknownStation' is an invalid name for this
example
    if station_name.strip().lower() == "unknownstation":
        return False
    return True # Assume valid for other names for now

def _parse_fare_response_new_api(response_data: dict) -> str:
    """
    Parses the NEW API's response and formats the fare details.
    !!! You MUST adapt this function based on the actual JSON
structure of your new API's response. !!!
    """
    if not response_data:
        return "No fare information available from the new API."

    fare_info = []
    # --- EXAMPLE PARSING - ADAPT THIS FOR YOUR NEW API'S RESPONSE ---
#
    # Assuming the new API returns something like {'fareAmount':
10.50, 'currency': 'USD', 'details': 'Direct route'}
    if "fareAmount" in response_data and "currency" in response_data:
        fare_info.append(f"  Base Fare: {response_data['fareAmount']}
{response_data['currency']}")
```

```
    if "travelTime" in response_data:
        fare_info.append(f"  Estimated Travel Time:
{response_data['travelTime']} minutes")
    if "routeDescription" in response_data:
        fare_info.append(f"  Route Description:
{response_data['routeDescription']}")

    if not fare_info:
        return "No detailed fare information found in the new API
response."

    return "\n".join(fare_info)

# --- Main Function for the NEW API --- #

def get_public_transport_fare_new_api(
    source_station: str, destination_station: str
) -> None:
    """
    Retrieves and displays public transport fare details between two
stations
    using the NEW API.
    Handles input validation, API calls, and error handling.
    """
    print(f"\n--- Retrieving fare for {source_station} to
{destination_station} using NEW API ---")

    # 1. Validate source and destination inputs using the new API's
validation logic
    if not _validate_station_name_new_api(source_station):
        print(f"Error: Invalid source station name for new API:
'{source_station}'")
        return
    if not _validate_station_name_new_api(destination_station):
        print(f"Error: Invalid destination station name for new API:
'{destination_station}'")
        return
    if source_station.lower() == destination_station.lower():
        print("Error: Source and destination stations cannot be the
same for new API.")
        return

    # Prepare API request parameters (ADAPT THESE FOR YOUR NEW API)
    params = {
        "origin": source_station, # New API might use 'origin' instead
of 'source'
        "destination": destination_station,
        "apiKey": NEW_API_KEY, # New API might use 'apiKey' instead of
'api_key'
    }
```

```
    headers = {
        "Accept": "application/json",
        # Add any other headers required by your NEW API
        # "Authorization": f"Bearer {NEW_API_KEY}"
    }

    try:
        # 2. Make API request to the NEW API
        print(f"Attempting to fetch fare from NEW API:
{NEW_API_ENDPOINT}...")
        response = requests.get(NEW_API_ENDPOINT, params=params,
headers=headers, timeout=10)
        response.raise_for_status()  # Raise HTTPError for bad
responses (4xx or 5xx)

        # Parse JSON response
        data = response.json()

        # 3. Handle NEW API specific errors
        # !!! ADAPT THESE CONDITIONS BASED ON HOW YOUR NEW API
INDICATES ERRORS !!!
        if response.status_code == 200:
            if data.get("status") == "error" or data.get("errorCode"):
                print(f"NEW API Error: {data.get('message',
data.get('errorDescription', 'Unknown API error.'))}")
            elif not data: # Check if response data is empty or
indicates no fare
                print("No fare information found for the given route
from NEW API. Check station names or route availability.")
            else:
                print(f"Fare details for {source_station} to
{destination_station} (NEW API):")
                print(_parse_fare_response_new_api(data)) # Pass the
entire data to the parser
        else:
            print(f"NEW API responded with status code
{response.status_code}: {data.get('message', 'No specific error
message from new API.')}")

    except requests.exceptions.HTTPError as e:
        print(f"NEW API HTTP Error: {e.response.status_code} -
{e.response.text}")
    except requests.exceptions.ConnectionError:
        print("NEW API Connection Error: Could not connect to the API.
Check internet/API availability.")
    except requests.exceptions.Timeout:
        print("NEW API Timeout Error: The API did not respond within
the expected time.")
    except requests.exceptions.RequestException as e:
```

```
        print(f"An unexpected request error occurred with NEW API:
{e}")
    except json.JSONDecodeError:
        print("Error: NEW API response was not a valid JSON.")
    except Exception as e:
        print(f"An unexpected error occurred with NEW API: {e}")

# --- Example Usage for the NEW API --- #
if __name__ == "__main__":
    get_public_transport_fare_new_api("StartStation", "EndStation")
    get_public_transport_fare_new_api("UnknownStation", "Destination")
# Invalid source
    get_public_transport_fare_new_api("City Center", "City Center") #
Same stations
```

```
--- Retrieving fare for StartStation to EndStation using NEW API ---
Attempting to fetch fare from NEW API: https://api.another-
transport.example.com/fare...
NEW API Connection Error: Could not connect to the API. Check
internet/API availability.

--- Retrieving fare for UnknownStation to Destination using NEW API
---
Error: Invalid source station name for new API: 'UnknownStation'

--- Retrieving fare for City Center to City Center using NEW API ---
Error: Source and destination stations cannot be the same for new API.
```

## Task 2: Currency Exchange Rates API Integration

**Task:** Develop a Python script to convert a given amount from INR to specified target currencies (USD, EUR, GBP) using an external API. The script must include AI-generated code, robust input validation for both the amount and currency codes, effective error handling for API failures, and clear, tabular output of conversion results.

```
import requests
import json

# --- Configuration for Currency Exchange API --- #
# !!! IMPORTANT: Replace with your actual Currency Exchange API
endpoint and API key !!!
# Example: https://www.exchangerate-api.com/, https://fixer.io/,
https://openexchangerates.org/
EXCHANGE_API_ENDPOINT = "https://api.exchangerate.host/latest"
EXCHANGE_API_KEY = "YOUR_EXCHANGE_API_KEY" # Some APIs might not need
a key for basic usage

# --- Helper Functions for Currency Exchange --- #
```

```python
def _validate_currency_code(currency_code: str) -> bool:
    """
    Validates if a given string is a valid 3-letter currency code.
    A real implementation might check against a comprehensive list of
ISO 4217 codes.
    """
    valid_currencies = ["INR", "USD", "EUR", "GBP"]
    return isinstance(currency_code, str) and currency_code.upper() in
valid_currencies

def _parse_exchange_response(response_data: dict, base_currency: str,
target_currencies: list) -> str:
    """
    Parses the API's response and formats the exchange rates into a
tabular string.
    !!! You MUST adapt this function based on the actual JSON
structure of your chosen API's response. !!!
    """
    if not response_data:
        return "No exchange rate information available from the API."

    conversion_info = []
    # --- EXAMPLE PARSING - ADAPT THIS FOR YOUR API'S RESPONSE --- #
    # This example assumes a structure like: {'rates': {'USD': 0.012,
'EUR': 0.011, 'GBP': 0.009}, 'base': 'INR'}

    rates = response_data.get("rates")
    if not rates:
        return "No rates found in the API response."

    # Prepare header
    conversion_info.append(f"\nCurrency Conversion from
{base_currency.upper()}:")
    conversion_info.append("----------------------------------------")
    conversion_info.append(f"{'Target Currency':<20} | {'Exchange
Rate':<15}")
    conversion_info.append("----------------------------------------")

    for target_currency in target_currencies:
        rate = rates.get(target_currency.upper())
        if rate is not None:
            conversion_info.append(f"{target_currency.upper():<20} |
{rate:<15.6f}")
        else:
            conversion_info.append(f"{target_currency.upper():<20} |
{'N/A':<15}")
    conversion_info.append("----------------------------------------")

    return "\n".join(conversion_info)
```

```python
# --- Main Function for Currency Exchange --- #

def get_currency_exchange_rates(
    amount_in_inr: float, target_currencies: list = None
) -> None:
    """
    Converts a given amount from INR to specified target currencies
using an API.
    Displays conversion results in a tabular format.
    """
    if target_currencies is None:
        target_currencies = ["USD", "EUR", "GBP"]

    print(f"\n--- Converting {amount_in_inr:.2f} INR to {',
'.join(tc.upper() for tc in target_currencies)} ---")

    # 1. Validate user input for amount
    if not isinstance(amount_in_inr, (int, float)) or amount_in_inr <=
0:
        print(f"Error: Invalid amount '{amount_in_inr}'. Amount must
be a positive number.")
        return

    # 1. Validate target currency codes
    for currency in target_currencies:
        if not _validate_currency_code(currency):
            print(f"Error: Invalid target currency code: '{currency}'.
Must be one of INR, USD, EUR, GBP.")
            return

    base_currency = "INR"

    # Prepare API request parameters (ADAPT THESE FOR YOUR API)
    params = {
        "base": base_currency,
        "symbols": ','.join([tc.upper() for tc in target_currencies]),
        # Some APIs require the API key as a query parameter
        # "access_key": EXCHANGE_API_KEY,
    }

    headers = {
        "Accept": "application/json",
        # Some APIs require the API key as an Authorization header
        # "Authorization": f"Bearer {EXCHANGE_API_KEY}"
    }

    try:
        # 2. Make API request
        print(f"Attempting to fetch exchange rates from:
{EXCHANGE_API_ENDPOINT}...")
```

```
        response = requests.get(EXCHANGE_API_ENDPOINT, params=params,
headers=headers, timeout=10)
        response.raise_for_status()  # Raise HTTPError for bad
responses (4xx or 5xx)

        # Parse JSON response
        data = response.json()

        # 3. Handle API specific errors
        # !!! ADAPT THESE CONDITIONS BASED ON HOW YOUR API INDICATES
ERRORS !!!
        if response.status_code == 200:
            if data.get("success") is False or data.get("error"):
                print(f"API Error: {data.get('error', {}).get('info',
'Unknown API error.')}")
            elif not data.get("rates"):
                print("No exchange rates found for the given
currencies. Check currency codes or API configuration.")
            else:
                print(_parse_exchange_response(data, base_currency,
target_currencies))
                print("\nConversion Results:")
                for currency in target_currencies:
                    rate = data.get("rates", {}).get(currency.upper())
                    if rate is not None:
                        converted_amount = amount_in_inr * rate
                        print(f"  {amount_in_inr:.2f}
{base_currency.upper()} = {converted_amount:.2f} {currency.upper()}")
                    else:
                        print(f"  Conversion to {currency.upper()} not
available.")
        else:
            print(f"API responded with status code
{response.status_code}: {data.get('error', 'No specific error
message.')}")

    except requests.exceptions.HTTPError as e:
        print(f"API HTTP Error: {e.response.status_code} -
{e.response.text}")
    except requests.exceptions.ConnectionError:
        print("Connection Error: Could not connect to the API. Check
internet/API availability.")
    except requests.exceptions.Timeout:
        print("Timeout Error: The API did not respond within the
expected time.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred with the Exchange
API: {e}")
    except json.JSONDecodeError:
        print("Error: Exchange API response was not a valid JSON.")
```

```
    except Exception as e:
        print(f"An unexpected error occurred with the Exchange API:
{e}")

# --- Example Usage for Currency Exchange --- #
if __name__ == "__main__":
    # Scenario 1: Valid conversion
    get_currency_exchange_rates(100.00)

    # Scenario 2: Valid conversion to specific currencies
    get_currency_exchange_rates(500.00, ["USD", "GBP"])

    # Scenario 3: Invalid amount
    get_currency_exchange_rates(-10.00)

    # Scenario 4: Invalid target currency
    get_currency_exchange_rates(200.00, ["USD", "XYZ"])

    # Scenario 5: Another valid conversion
    get_currency_exchange_rates(1000.00, ["USD", "EUR", "GBP"])
```

```
--- Converting 100.00 INR to USD, EUR, GBP ---
Attempting to fetch exchange rates from:
https://api.exchangerate.host/latest...
API Error: You have not supplied an API Access Key. [Required format:
access_key=YOUR_ACCESS_KEY]

--- Converting 500.00 INR to USD, GBP ---
Attempting to fetch exchange rates from:
https://api.exchangerate.host/latest...
API Error: You have not supplied an API Access Key. [Required format:
access_key=YOUR_ACCESS_KEY]

--- Converting -10.00 INR to USD, EUR, GBP ---
Error: Invalid amount '-10.0'. Amount must be a positive number.

--- Converting 200.00 INR to USD, XYZ ---
Error: Invalid target currency code: 'XYZ'. Must be one of INR, USD,
EUR, GBP.

--- Converting 1000.00 INR to USD, EUR, GBP ---
Attempting to fetch exchange rates from:
https://api.exchangerate.host/latest...
API Error: You have not supplied an API Access Key. [Required format:
access_key=YOUR_ACCESS_KEY]
```

## Task 3: GitHub Repository Info Fetcher

**Task:** Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

**Instructions:**

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

**Expected Output:** A Python program that retrieves and displays GitHub repository information with robust error handling.

```
import requests
import json

# --- Configuration for GitHub API --- #
# Personal Access Token (PAT) is recommended for higher rate limits.
# Create one at: https://github.com/settings/tokens (select 'repo'
scope if needed for private repos)
GITHUB_API_BASE_URL = "https://api.github.com"
GITHUB_TOKEN = "YOUR_GITHUB_PERSONAL_ACCESS_TOKEN" # Optional, but
recommended for higher rate limits

# --- Helper Function for GitHub API --- #

def _parse_github_repo_response(repo_data: dict) -> str:
    """
    Parses the GitHub API response for repository details and formats
them.
    """
    if not repo_data:
        return "No repository data received."

    repo_info = []
    repo_info.append(f"Repository Name: {repo_data.get('name',
'N/A')}")
    repo_info.append(f"Description: {repo_data.get('description', 'No
description provided.')}")
    repo_info.append(f"Stars: {repo_data.get('stargazers_count', 0)}")
    repo_info.append(f"Forks: {repo_data.get('forks_count', 0)}")
    repo_info.append(f"Open Issues:
{repo_data.get('open_issues_count', 0)}")
    repo_info.append(f"URL: {repo_data.get('html_url', 'N/A')}")
    repo_info.append(f"Default Branch:
{repo_data.get('default_branch', 'N/A')}")

    return "\n".join(repo_info)

# --- Main Function for GitHub Repository Info --- #

def get_github_repo_info(owner: str, repo_name: str) -> None:
    """
    Fetches and displays GitHub repository details (stars, forks,
```

```
issues).
    Handles rate limit errors, 404 (repo not found), and invalid
input.
    """
    print(f"\n--- Fetching info for GitHub repository:
{owner}/{repo_name} ---")

    # 1. Validate input
    if not isinstance(owner, str) or not owner.strip():
        print("Error: Owner name cannot be empty.")
        return
    if not isinstance(repo_name, str) or not repo_name.strip():
        print("Error: Repository name cannot be empty.")
        return

    # Construct the API URL
    url = f"{GITHUB_API_BASE_URL}/repos/{owner}/{repo_name}"

    headers = {
        "Accept": "application/vnd.github.v3+json", # Recommended for
GitHub API
    }
    # Add Authorization header if a token is provided
    if GITHUB_TOKEN and GITHUB_TOKEN !=
"YOUR_GITHUB_PERSONAL_ACCESS_TOKEN":
        headers["Authorization"] = f"token {GITHUB_TOKEN}"
    else:
        print("Warning: No GitHub Personal Access Token provided. Rate
limits might be lower.")

    try:
        # 2. Make API request
        print(f"Attempting to fetch from: {url}...")
        response = requests.get(url, headers=headers, timeout=10)
        response.raise_for_status()  # Raise HTTPError for bad
responses (4xx or 5xx)

        # Parse JSON response
        data = response.json()

        # 3. Display repository information
        print(_parse_github_repo_response(data))

    except requests.exceptions.HTTPError as e:
        if e.response.status_code == 404:
            print(f"Error: Repository '{owner}/{repo_name}' not found.
Check owner and repository name.")
        elif e.response.status_code == 403 and 'X-RateLimit-Remaining'
in e.response.headers and int(e.response.headers['X-RateLimit-
Remaining']) == 0:
```

```
            print(f"Error: GitHub API rate limit exceeded. Try again
later. Resets at {e.response.headers.get('X-RateLimit-Reset')}
(UTC).")
        else:
            print(f"GitHub API HTTP Error: {e.response.status_code} -
{e.response.text}")
    except requests.exceptions.ConnectionError:
        print("Connection Error: Could not connect to GitHub API.
Check internet connection or API availability.")
    except requests.exceptions.Timeout:
        print("Timeout Error: GitHub API did not respond within the
expected time.")
    except requests.exceptions.RequestException as e:
        print(f"An unexpected request error occurred with GitHub API:
{e}")
    except json.JSONDecodeError:
        print("Error: GitHub API response was not a valid JSON.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

# --- Example Usage for GitHub Repo Info --- #
if __name__ == "__main__":
    # Scenario 1: Valid public repository
    get_github_repo_info("octocat", "Spoon-Knife")

    # Scenario 2: Another valid public repository
    get_github_repo_info("pallets", "flask")

    # Scenario 3: Repository not found (404 error)
    get_github_repo_info("nonexistent-user", "nonexistent-repo-123")

    # Scenario 4: Invalid input - empty owner
    get_github_repo_info("", "some-repo")

    # Scenario 5: Invalid input - empty repo name
    get_github_repo_info("github", "")

    # Scenario 6: Test rate limit (you might need to make many
requests rapidly to hit this)
    # get_github_repo_info("kubernetes", "kubernetes")


--- Fetching info for GitHub repository: octocat/Spoon-Knife ---
Warning: No GitHub Personal Access Token provided. Rate limits might
be lower.
Attempting to fetch from: https://api.github.com/repos/octocat/Spoon-
Knife...
Repository Name: Spoon-Knife
Description: This repo is for demonstration purposes only.
Stars: 13707
```

```
Forks: 156796
Open Issues: 20452
URL: https://github.com/octocat/Spoon-Knife
Default Branch: main

--- Fetching info for GitHub repository: pallets/flask ---
Warning: No GitHub Personal Access Token provided. Rate limits might
be lower.
Attempting to fetch from:
https://api.github.com/repos/pallets/flask...
Repository Name: flask
Description: The Python micro framework for building web applications.
Stars: 71362
Forks: 16760
Open Issues: 3
URL: https://github.com/pallets/flask
Default Branch: main

--- Fetching info for GitHub repository: nonexistent-user/nonexistent-
repo-123 ---
Warning: No GitHub Personal Access Token provided. Rate limits might
be lower.
Attempting to fetch from: https://api.github.com/repos/nonexistent-
user/nonexistent-repo-123...
Error: Repository 'nonexistent-user/nonexistent-repo-123' not found.
Check owner and repository name.

--- Fetching info for GitHub repository: /some-repo ---
Error: Owner name cannot be empty.

--- Fetching info for GitHub repository: github/ ---
Error: Repository name cannot be empty.
```

## Task 4: Real-Time Application: News Headlines Aggregator

**Scenario:** Build a News Aggregator using a free News API.

**Requirements:**

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

**Expected Output:** A working Python script that retrieves and displays news headlines with proper error handling.

```python
import requests
import json
```

```python
import time

# --- Configuration for News API --- #
# !!! IMPORTANT: Replace with your actual News API endpoint and API
key !!!
# Example: https://newsapi.org/docs/endpoints/top-headlines
NEWS_API_ENDPOINT = "https://newsapi.org/v2/top-headlines"
NEWS_API_KEY = "YOUR_NEWS_API_KEY" # Get a free API key from
NewsAPI.org or similar

# --- Helper Functions for News Aggregator --- #

def _validate_category(category: str) -> bool:
    """
    Validates if a given category is supported by the news API.
    """
    supported_categories = ["business", "entertainment", "general",
"health", "science", "sports", "technology"]
    return category.lower() in supported_categories

def _parse_headlines_response(response_data: dict, num_headlines: int
= 5) -> str:
    """
    Parses the News API's response and formats the top headlines.
    !!! You MUST adapt this function based on the actual JSON
structure of your chosen API's response. !!!
    """
    if not response_data or not response_data.get('articles'):
        return "No headlines found for the given category."

    headlines_info = ["Top Headlines:"]
    articles = response_data['articles']

    for i, article in enumerate(articles[:num_headlines]):
        title = article.get('title', 'No Title')
        source = article.get('source', {}).get('name', 'Unknown
Source')
        url = article.get('url', '#')
        headlines_info.append(f"{i+1}. {title} ({source})\n   Link:
{url}")

    return "\n".join(headlines_info)

def _get_headlines_with_retry(category: str, retries: int = 3, delay:
int = 2) -> dict or None:
    """
    Attempts to fetch headlines with a retry mechanism.
    """
    for attempt in range(retries):
        try:
```

```
            params = {
                "category": category,
                "pageSize": 5, # Fetch 5 headlines as per requirement
                "apiKey": NEWS_API_KEY,
                "language": "en" # Example: specify language
            }
            headers = {
                "Accept": "application/json"
            }

            print(f"Attempt {attempt + 1}/{retries}: Fetching
headlines for category '{category}'...")
            response = requests.get(NEWS_API_ENDPOINT, params=params,
headers=headers, timeout=10)
            response.raise_for_status() # Raise HTTPError for bad
responses (4xx or 5xx)

            data = response.json()

            # NewsAPI.org specific error handling for API key issues
            if data.get('status') == 'error':
                error_code = data.get('code')
                error_message = data.get('message')
                if error_code == 'apiKeyMissing':
                    print(f"Error: News API Key is missing. Please
replace 'YOUR_NEWS_API_KEY' with your actual key.")
                    return None
                elif error_code == 'apiKeyInvalid':
                    print(f"Error: News API Key is invalid. Please
check your API key.")
                    return None
                elif error_code == 'categoryInvalid':
                     print(f"Error: Invalid category '{category}'
according to News API. Please choose from supported categories.")
                     return None
                else:
                    print(f"News API Error ({error_code}):
{error_message}")
                    return None

            return data

        except requests.exceptions.HTTPError as e:
            if e.response.status_code == 401: # Unauthorized - likely
API key issue
                print(f"HTTP Error 401: Unauthorized. Please check
your News API Key. (Attempt {attempt + 1})")
                return None # No point retrying with bad key
            elif e.response.status_code == 429: # Too Many Requests -
Rate Limit
```

```
                print(f"HTTP Error 429: Rate limit exceeded. Retrying
in {delay} seconds... (Attempt {attempt + 1})")
            else:
                print(f"HTTP Error: {e.response.status_code} -
{e.response.text} (Attempt {attempt + 1})")
        except requests.exceptions.ConnectionError:
            print(f"Connection Error: Could not connect to News API.
Retrying in {delay} seconds... (Attempt {attempt + 1})")
        except requests.exceptions.Timeout:
            print(f"Timeout Error: News API did not respond in time.
Retrying in {delay} seconds... (Attempt {attempt + 1})")
        except json.JSONDecodeError:
            print(f"Error: News API response was not a valid JSON.
(Attempt {attempt + 1})")
        except Exception as e:
            print(f"An unexpected error occurred during API call: {e}
(Attempt {attempt + 1})")

        if attempt < retries - 1:
            time.sleep(delay)

    print(f"Failed to fetch headlines after {retries} attempts.")
    return None

# --- Main Function for News Aggregator --- #

def get_news_headlines(category: str) -> None:
    """
    Fetches and displays top 5 news headlines for a given category.
    Includes input validation, API call with retry, and error
handling.
    """
    print(f"\n--- Fetching News Headlines for Category:
{category.upper()} ---")

    # 1. Validate category input
    if not isinstance(category, str) or not category.strip():
        print("Error: Category cannot be empty.")
        return
    if not _validate_category(category):
        print(f"Error: Invalid category '{category}'. Supported
categories are: business, entertainment, general, health, science,
sports, technology.")
        return

    # Ensure API Key is not default placeholder
    if NEWS_API_KEY == "YOUR_NEWS_API_KEY":
        print("Error: News API Key is not configured. Please replace
'YOUR_NEWS_API_KEY' with your actual API key.")
        return
```

```
    # 2. Fetch headlines with retry mechanism
    news_data = _get_headlines_with_retry(category.lower())

    # 3. Display results or error message
    if news_data:
        print(_parse_headlines_response(news_data, num_headlines=5))
    else:
        print("Could not retrieve news headlines. Please check
configuration and try again.")

# --- Example Usage for News Aggregator --- #
if __name__ == "__main__":
    # Scenario 1: Valid category (e.g., technology)
    get_news_headlines("technology")

    # Scenario 2: Another valid category (e.g., sports)
    get_news_headlines("sports")

    # Scenario 3: Invalid category
    get_news_headlines("cooking")

    # Scenario 4: Empty category
    get_news_headlines("")

    # To test API key errors, either leave NEWS_API_KEY as default or
intentionally set it wrong.
    # To test retry, you might need to artificially make the first
request fail (e.g., wrong endpoint for 1 attempt).
```

```
--- Fetching News Headlines for Category: TECHNOLOGY ---
Error: News API Key is not configured. Please replace
'YOUR_NEWS_API_KEY' with your actual API key.

--- Fetching News Headlines for Category: SPORTS ---
Error: News API Key is not configured. Please replace
'YOUR_NEWS_API_KEY' with your actual API key.

--- Fetching News Headlines for Category: COOKING ---
Error: Invalid category 'cooking'. Supported categories are: business,
entertainment, general, health, science, sports, technology.

--- Fetching News Headlines for Category:  ---
Error: Category cannot be empty.
```

## Task 5: COVID-19 Statistics API Integration

**Task:** Connect to a COVID-19 Statistics API to fetch country-wise COVID data.

**Instructions:**

- Use AI to generate Python code using the requests library.
- Accept country name as user input.
- Fetch key statistics such as:
  - Total confirmed cases
  - Total deaths
  - Total recovered cases
  - Active cases
- Handle API errors including:
  - Invalid country name
  - API rate-limit exceeded
  - Network or server failures
- Implement retry or wait logic when rate limits are hit.

**Expected Output:** A Python program that retrieves and displays country-wise COVID-19 statistics with proper rate-limit handling and error management.

```
import requests
import json
import time

# --- Configuration for COVID-19 Statistics API --- #
# Using disease.sh API as a free and comprehensive option.
# Docs: https://disease.sh/docs/
COVID_API_BASE_URL = "https://disease.sh/v3/covid-19"

# --- Helper Functions for COVID-19 Statistics --- #

def _parse_covid_response(country_data: dict, country_name: str) ->
str:
    """
    Parses the COVID-19 API's response and formats the statistics.
    """
    if not country_data:
        return f"No COVID-19 data available for {country_name}."

    stats_info = [f"COVID-19 Statistics for {country_name.title()}:"]
    stats_info.append("----------------------------------------")
    stats_info.append(f"Total Confirmed: {country_data.get('cases',
'N/A'):,}")
    stats_info.append(f"Total Deaths:    {country_data.get('deaths',
'N/A'):,}")
    stats_info.append(f"Total Recovered:
{country_data.get('recovered', 'N/A'):,}")
    stats_info.append(f"Active Cases:    {country_data.get('active',
'N/A'):,}")
    stats_info.append("----------------------------------------")

    return "\n".join(stats_info)
```

```
def _get_country_data_with_retry(country: str, retries: int = 3,
delay: int = 5) -> dict or None:
    """
    Attempts to fetch country-wise COVID-19 data with a retry
mechanism.
    """
    url = f"{COVID_API_BASE_URL}/countries/{country}"

    for attempt in range(retries):
        print(f"Attempt {attempt + 1}/{retries}: Fetching COVID-19
data for '{country}'...")
        try:
            response = requests.get(url, timeout=10)
            response.raise_for_status()  # Raise HTTPError for bad
responses (4xx or 5xx)

            data = response.json()

            # disease.sh returns an 'error' field and 404 status for
invalid countries
            if response.status_code == 404 and data.get('message') ==
'Country not found or doesn\'t have any cases':
                print(f"Error: Country '{country}' not found or no
data available. Please check the spelling.")
                return None # No point in retrying for a clearly
invalid country

            return data

        except requests.exceptions.HTTPError as e:
            if e.response.status_code == 429: # Too Many Requests -
Rate Limit
                print(f"HTTP Error 429: API rate limit exceeded.
Retrying in {delay} seconds... (Attempt {attempt + 1})")
            elif e.response.status_code == 404:
                # This case is handled above specifically for
disease.sh's error message
                print(f"HTTP Error 404: {country} not found. (Attempt
{attempt + 1})")
                return None # Stop if country is definitely not found
            else:
                print(f"HTTP Error: {e.response.status_code} -
{e.response.text} (Attempt {attempt + 1})")
        except requests.exceptions.ConnectionError:
            print(f"Connection Error: Could not connect to COVID-19
API. Retrying in {delay} seconds... (Attempt {attempt + 1})")
        except requests.exceptions.Timeout:
            print(f"Timeout Error: The API did not respond within the
expected time. Retrying in {delay} seconds... (Attempt {attempt +
```

```
1})")
        except json.JSONDecodeError:
            print(f"Error: API response was not a valid JSON. (Attempt
{attempt + 1})")
        except Exception as e:
            print(f"An unexpected error occurred during API call: {e}
(Attempt {attempt + 1})")

        if attempt < retries - 1: # Don't sleep after the last attempt
            time.sleep(delay)

    print(f"Failed to fetch COVID-19 data for '{country}' after
{retries} attempts.")
    return None

# --- Main Function for COVID-19 Statistics --- #

def get_covid_statistics(country_name: str) -> None:
    """
    Fetches and displays country-wise COVID-19 statistics.
    Handles input validation, API calls with retry, and error
management.
    """
    print(f"\n--- Fetching COVID-19 Statistics for:
{country_name.title()} ---")

    # 1. Validate country input
    if not isinstance(country_name, str) or not country_name.strip():
        print("Error: Country name cannot be empty.")
        return

    # 2. Fetch data with retry mechanism
    covid_data = _get_country_data_with_retry(country_name.strip())

    # 3. Display results or error message
    if covid_data:
        print(_parse_covid_response(covid_data, country_name))
    else:
        print(f"Could not retrieve COVID-19 statistics for
{country_name}. Please check the country name or try again later.")

# --- Example Usage for COVID-19 Statistics --- #
if __name__ == "__main__":
    # Scenario 1: Valid country
    get_covid_statistics("India")
    get_covid_statistics("United States")

    # Scenario 2: Country not found
    get_covid_statistics("Wakanda")
    get_covid_statistics("Atlantis")
```

```
    # Scenario 3: Empty input
    get_covid_statistics("")

    # Scenario 4: Country with space (should be handled by strip() and
API)
    get_covid_statistics(" United Kingdom ")

    # To test rate limiting, you might need to make many requests very
quickly
    # For example:
    # for _ in range(50): # This might trigger rate limit depending on
API policy
    #     get_covid_statistics("Canada")
```

```
--- Fetching COVID-19 Statistics for: India ---
Attempt 1/3: Fetching COVID-19 data for 'India'...
COVID-19 Statistics for India:
----------------------------------------
Total Confirmed: 45,035,393
Total Deaths:    533,570
Total Recovered: 0
Active Cases:    44,501,823
----------------------------------------

--- Fetching COVID-19 Statistics for: United States ---
Attempt 1/3: Fetching COVID-19 data for 'United States'...
COVID-19 Statistics for United States:
----------------------------------------
Total Confirmed: 111,820,082
Total Deaths:    1,219,487
Total Recovered: 109,814,428
Active Cases:    786,167
----------------------------------------

--- Fetching COVID-19 Statistics for: Wakanda ---
Attempt 1/3: Fetching COVID-19 data for 'Wakanda'...
HTTP Error 404: Wakanda not found. (Attempt 1)
Could not retrieve COVID-19 statistics for Wakanda. Please check the
country name or try again later.

--- Fetching COVID-19 Statistics for: Atlantis ---
Attempt 1/3: Fetching COVID-19 data for 'Atlantis'...
HTTP Error 404: Atlantis not found. (Attempt 1)
Could not retrieve COVID-19 statistics for Atlantis. Please check the
country name or try again later.

--- Fetching COVID-19 Statistics for:  ---
Error: Country name cannot be empty.
```

```
--- Fetching COVID-19 Statistics for:  United Kingdom  ---
Attempt 1/3: Fetching COVID-19 data for 'United Kingdom'...
COVID-19 Statistics for  United Kingdom :
----------------------------------------
Total Confirmed: 24,910,387
Total Deaths:    232,112
Total Recovered: 24,678,275
Active Cases:    0
----------------------------------------
```